Smart Farming - Crop Yield Prediction using IoT and Satellite Data

Introduction

This project explores a real-world inspired dataset simulating smart farming operations across 500 farms located in regions such as India, the USA, and Africa. The dataset integrates data collected from IoT sensors and satellite observations to help analyze environmental and operational factors that influence crop yield.

The ultimate goal is to build a high-value data science project that:

- Explores the relationship between environmental factors and crop yield.
- Performs advanced data cleaning and exploratory analysis.
- Uses machine learning to predict yield based on various features.
- Provides actionable insights into smart agriculture optimization.

Dataset Highlights

- **Regions:** Multiple continents (India, USA, Africa)
- **Variables:** Soil moisture, temperature, humidity, NDVI, irrigation type, crop disease status, etc.
- **Output variable:** `yield_kg_per_hectare`
- **Use cases:** Predictive modeling, time-series analysis, spatial data visualization, sensor optimization

Project Workflow

1. **Data Loading & Cleaning**
2. **Exploratory Data Analysis (EDA)**
3. **Feature Engineering**
4. **Predictive Modeling**
5. **Advanced Visualizations**
6. **Sensor & Time Trends**
7. **Conclusion & Recommendations**

Phase 1: Data Cleaning + Exploratory Data Analysis (EDA)

In [1]:
```
# Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import plotly.express as px
```

In [2]:
```
# Setting display options
pd.set_option('display.max_columns', None)
```

In [3]:
```
# Loading Dataset
df = pd.read_csv("Crop_Prediction.csv")
# Displaying the first few rows of the dataset
df.head()
```

Out[3]:

	farm_id	region	crop_type	soil_moisture_%	soil_pH	temperature_C	rainfall_mm
0	FARM0001	North India	Wheat	35.95	5.99	17.79	75.62
1	FARM0002	South USA	Soybean	19.74	7.24	30.18	89.91
2	FARM0003	South USA	Wheat	29.32	7.16	27.37	265.43
3	FARM0004	Central USA	Maize	17.33	6.03	33.73	212.01
4	FARM0005	Central USA	Cotton	19.37	5.92	33.86	269.09

In [4]:
```
# Basic Info
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   farm_id               500 non-null    object
 1   region                500 non-null    object
 2   crop_type             500 non-null    object
 3   soil_moisture_%       500 non-null    float64
 4   soil_pH               500 non-null    float64
 5   temperature_C         500 non-null    float64
 6   rainfall_mm           500 non-null    float64
 7   humidity_%            500 non-null    float64
 8   sunlight_hours        500 non-null    float64
 9   irrigation_type       350 non-null    object
 10  fertilizer_type       500 non-null    object
 11  pesticide_usage_ml    500 non-null    float64
 12  sowing_date           500 non-null    object
 13  harvest_date          500 non-null    object
 14  total_days            500 non-null    int64
 15  yield_kg_per_hectare  500 non-null    float64
 16  sensor_id             500 non-null    object
 17  timestamp             500 non-null    object
 18  latitude              500 non-null    float64
 19  longitude             500 non-null    float64
 20  NDVI_index            500 non-null    float64
 21  crop_disease_status   370 non-null    object
dtypes: float64(11), int64(1), object(10)
memory usage: 86.1+ KB
```

Out[4]:

	soil_moisture_%	soil_pH	temperature_C	rainfall_mm	humidity_%	sunlight_h
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.0
mean	26.750140	6.523980	24.675740	181.685740	65.194460	7.0
std	10.150053	0.585558	5.348899	72.293091	14.642849	1.6
min	10.160000	5.510000	15.000000	50.170000	40.230000	4.0
25%	17.890000	6.030000	20.295000	119.217500	51.865000	5.6
50%	25.855000	6.530000	24.655000	191.545000	65.685000	6.9
75%	36.022500	7.040000	29.090000	239.035000	77.995000	8.4
max	44.980000	7.500000	34.840000	298.960000	90.000000	10.0

Dataset Summary

This dataset contains **500 farm records** with **22 features** related to crop growth, environmental conditions, and yield. It is suitable for tasks like **crop yield prediction**, **disease detection**, and **smart farming analytics**.

🔹 Key Features:

- **Numerical Features:**

- `soil_moisture_%` : Mean = 26.75%, Range = 10.16 – 44.98
- `soil_pH` : Mean = 6.52, nearly neutral
- `temperature_C` : Mean = 24.68°C, Range = 15 – 34.84°C
- `rainfall_mm` : Mean = 181.68 mm
- `humidity_%` : Mean = 65.19%
- `sunlight_hours` : Mean = 7 hours/day
- `pesticide_usage_ml` : Mean = 26.59 ml
- `yield_kg_per_hectare` : Mean = 4032.93 kg/ha
- `NDVI_index` : Mean = 0.60 (crop health indicator)

- **Categorical Features:**
  - `crop_type` , `region` , `irrigation_type` , `fertilizer_type` , `crop_disease_status`
- **Temporal Features:**
  - `sowing_date` , `harvest_date` : Used to calculate `total_days`
  - `timestamp` : Sensor data recording time
- **Geospatial Features:**
  - `latitude` , `longitude` : Farm locations for mapping and spatial analysis

Insights:

- **Yield** ranges from ~2000 to ~6000 kg/ha — a key variable for prediction models.
- **NDVI**, weather, and soil conditions are likely strong predictors.
- Minimal missing data, good for training ML models.

In [6]:
```
# Checking Missing Values
print("Missing Values:\n")
print(df.isnull().sum())
```

```
Missing Values:

farm_id                   0
region                    0
crop_type                 0
soil_moisture_%           0
soil_pH                   0
temperature_C             0
rainfall_mm               0
humidity_%                0
sunlight_hours            0
irrigation_type         150
fertilizer_type           0
pesticide_usage_ml        0
sowing_date               0
harvest_date              0
total_days                0
yield_kg_per_hectare      0
sensor_id                 0
timestamp                 0
latitude                  0
longitude                 0
NDVI_index                0
crop_disease_status     130
dtype: int64
```

Missing values: - `irrigation_type` : 150 missing - `crop_disease_status` : 130 missing

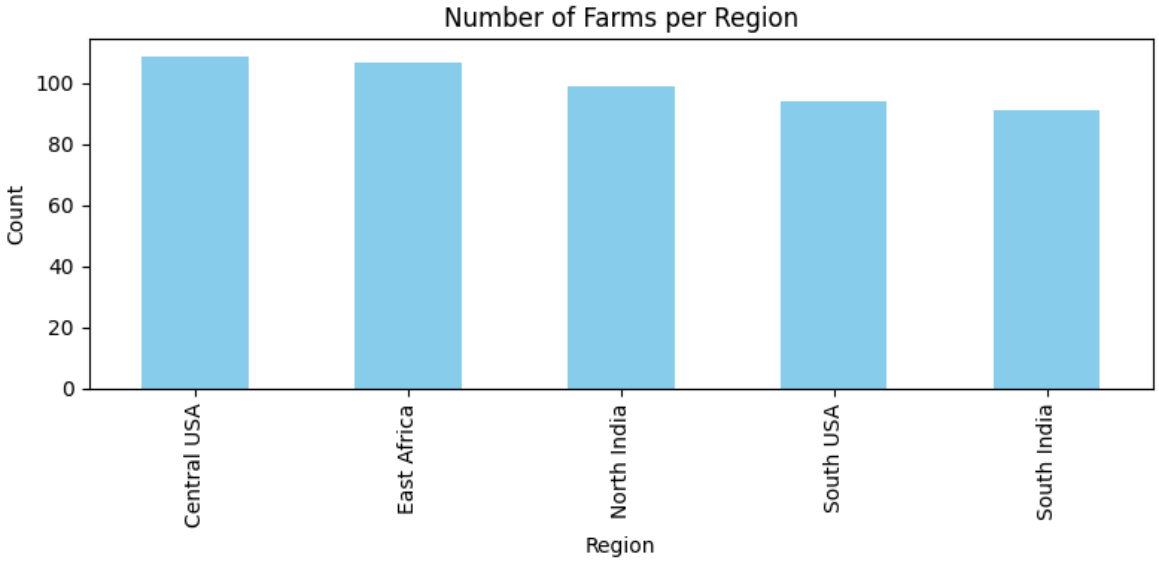
In [7]:
```
# Filling missing irrigation_type with 'Unknown'
df['irrigation_type'] = df['irrigation_type'].fillna('Unknown')

# Filling missing crop_disease_status with 'None' (assuming no symptoms observed
df['crop_disease_status'] = df['crop_disease_status'].fillna('None')

# Recreating disease_severity after imputation
disease_map = {'None': 0, 'Mild': 1, 'Severe': 2}
df['disease_severity'] = df['crop_disease_status'].map(disease_map)
```

In [8]:
```
# Cleaning: Converting Dates to datetime
df['sowing_date'] = pd.to_datetime(df['sowing_date'])
df['harvest_date'] = pd.to_datetime(df['harvest_date'])
df['timestamp'] = pd.to_datetime(df['timestamp'])
```

In [9]:
```
# Region-wise Farm Count
plt.figure(figsize=(8,4))
df['region'].value_counts().plot(kind='bar', color='skyblue')
plt.title('Number of Farms per Region')
plt.xlabel('Region')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

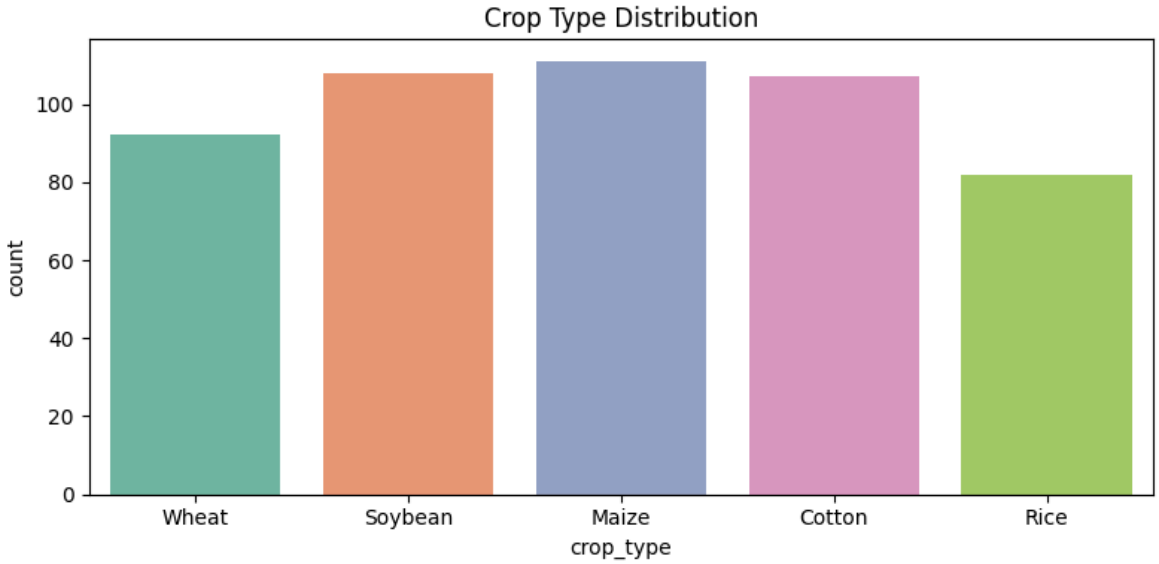


In [10]:
```
# Crop Type Count
plt.figure(figsize=(8,4))
sns.countplot(data=df, x='crop_type', palette='Set2')
plt.title('Crop Type Distribution')
plt.tight_layout()
plt.show()
```

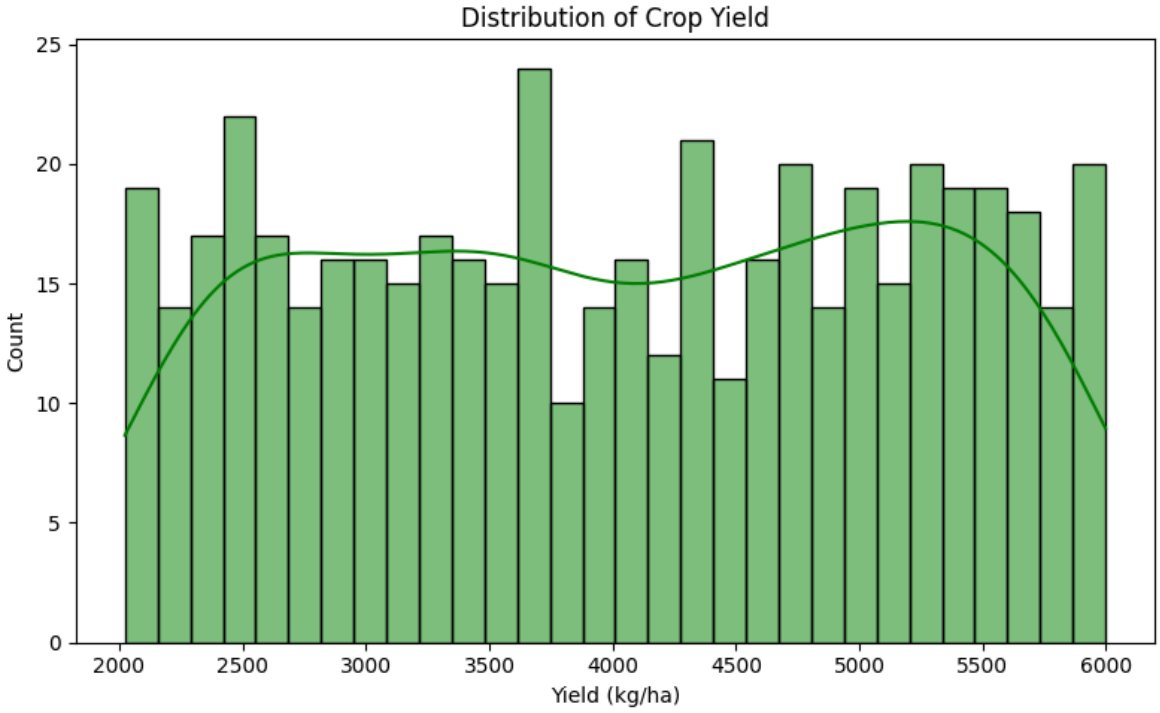
```
C:\Users\cuty computey\AppData\Local\Temp\ipykernel_7316\3987280416.py:3: FutureW
arning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.countplot(data=df, x='crop_type', palette='Set2')
```



In [11]:
```
# Yield Distribution
plt.figure(figsize=(8,5))
sns.histplot(df['yield_kg_per_hectare'], bins=30, kde=True, color='green')
plt.title('Distribution of Crop Yield')
plt.xlabel('Yield (kg/ha)')
plt.tight_layout()
plt.show()
```

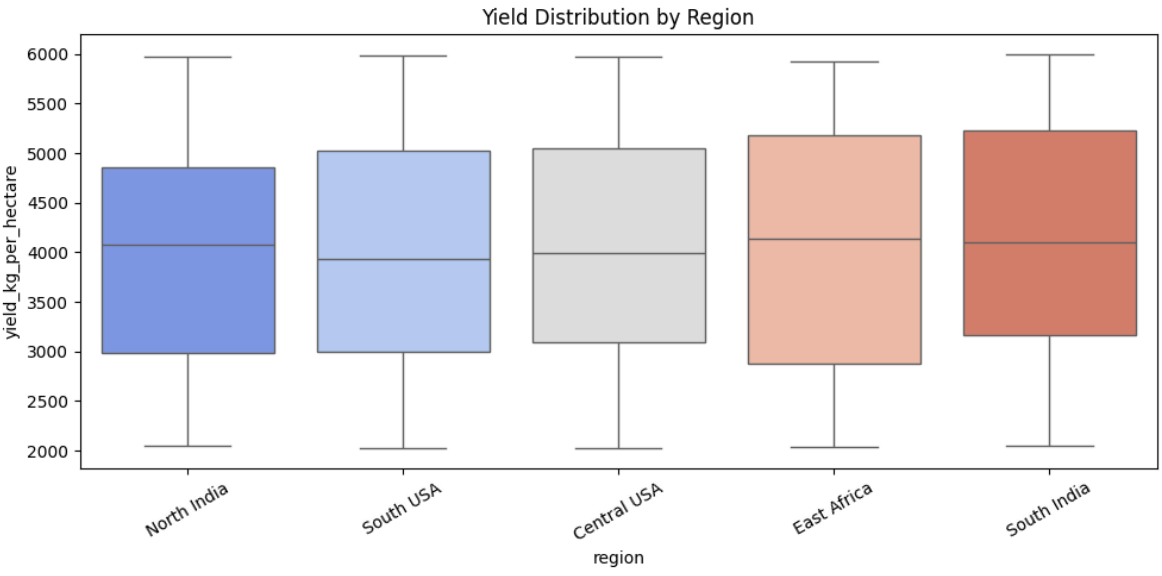


In [12]:
```
# Yield by Region
plt.figure(figsize=(10,5))
sns.boxplot(data=df, x='region', y='yield_kg_per_hectare', palette='coolwarm')
plt.title('Yield Distribution by Region')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```

```
C:\Users\cuty computey\AppData\Local\Temp\ipykernel_7316\3273994108.py:3: FutureW
arning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.boxplot(data=df, x='region', y='yield_kg_per_hectare', palette='coolwarm')
```



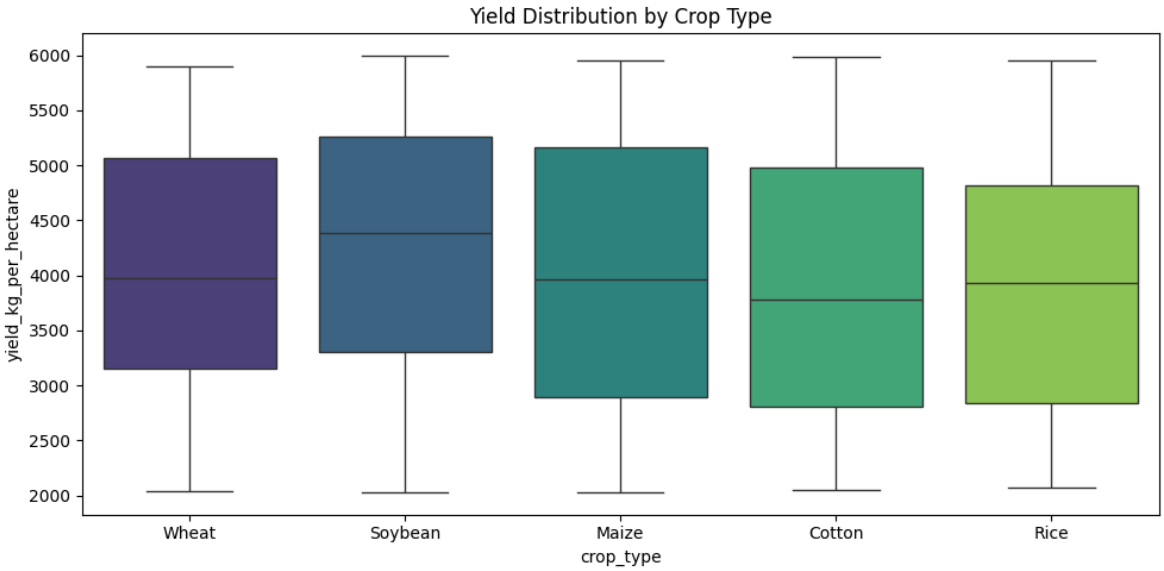
In [13]:
```
# Yield by Crop Type
plt.figure(figsize=(10,5))
sns.boxplot(data=df, x='crop_type', y='yield_kg_per_hectare', palette='viridis')
```

```
plt.title('Yield Distribution by Crop Type')
plt.tight_layout()
plt.show()
```

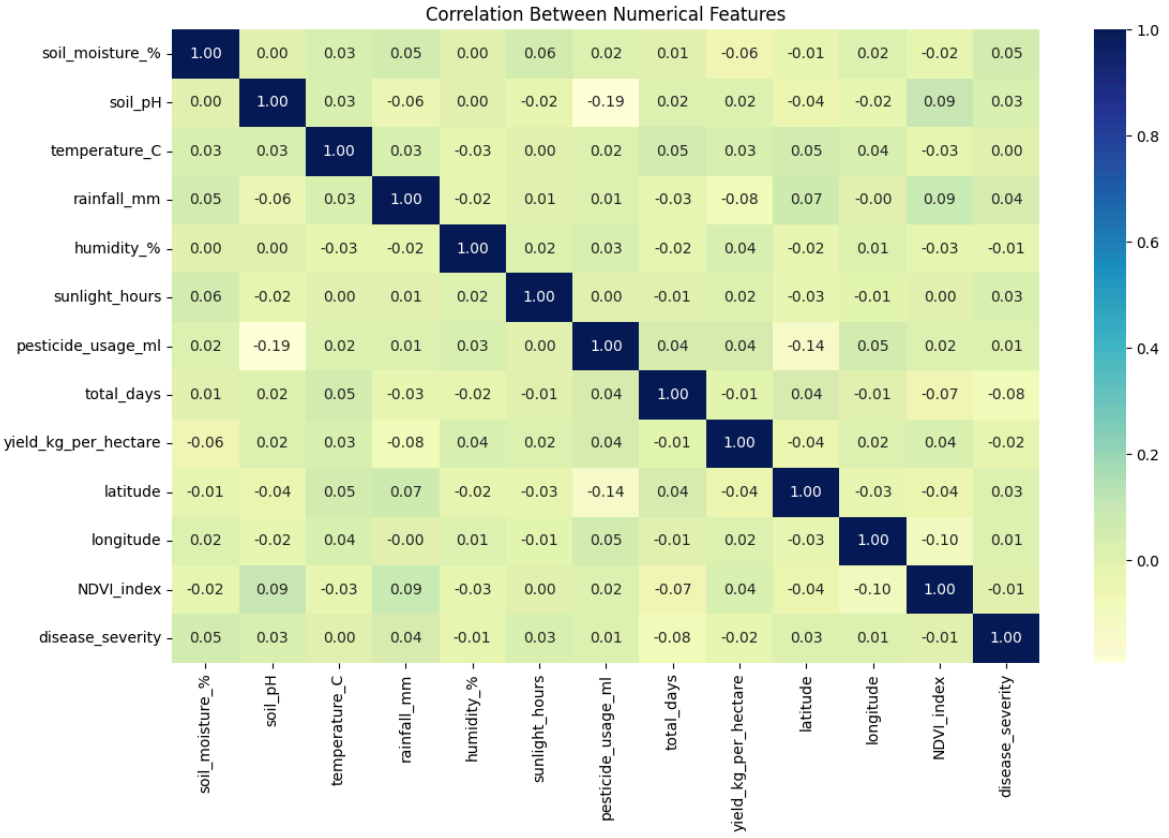
```
C:\Users\cuty computey\AppData\Local\Temp\ipykernel_7316\4076688982.py:3: FutureW
arning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.boxplot(data=df, x='crop_type', y='yield_kg_per_hectare', palette='viridi
s')
```



In [14]:
```
# Correlation Heatmap (numeric features)
numeric_cols = df.select_dtypes(include=['float64', 'int64'])
plt.figure(figsize=(12,8))
sns.heatmap(numeric_cols.corr(), annot=True, cmap='YlGnBu', fmt='.2f')
plt.title('Correlation Between Numerical Features')
plt.tight_layout()
plt.show()
```



Phase 2: Feature Engineering

In [15]:
```
# Creating New Features

# 1. Encoding crop_disease_status to numeric severity score
disease_map = {'None': 0, 'Mild': 1, 'Severe': 2}
df['disease_severity'] = df['crop_disease_status'].map(disease_map)

# 2. Extracting season from sowing_date
df['sowing_month'] = df['sowing_date'].dt.month

def get_season(month):
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Autumn'
```

In [16]:
```
# Applying season label
df['sowing_season'] = df['sowing_month'].apply(get_season)

# 3. Encode categorical features using Label Encoding
from sklearn.preprocessing import LabelEncoder

label_cols = ['region', 'crop_type', 'irrigation_type', 'fertilizer_type', 'sowi
label_encoders = {}

for col in label_cols:
    le = LabelEncoder()
```

```
    df[col + '_encoded'] = le.fit_transform(df[col])
    label_encoders[col] = le
```

In [17]:
```
# Final feature list
selected_features = [
    'soil_moisture_%', 'soil_pH', 'temperature_C', 'rainfall_mm', 'humidity_%',
    'sunlight_hours', 'pesticide_usage_ml', 'total_days', 'NDVI_index', 'disease
    'region_encoded', 'crop_type_encoded', 'irrigation_type_encoded', 'fertilize
    'sowing_season_encoded'
]
```

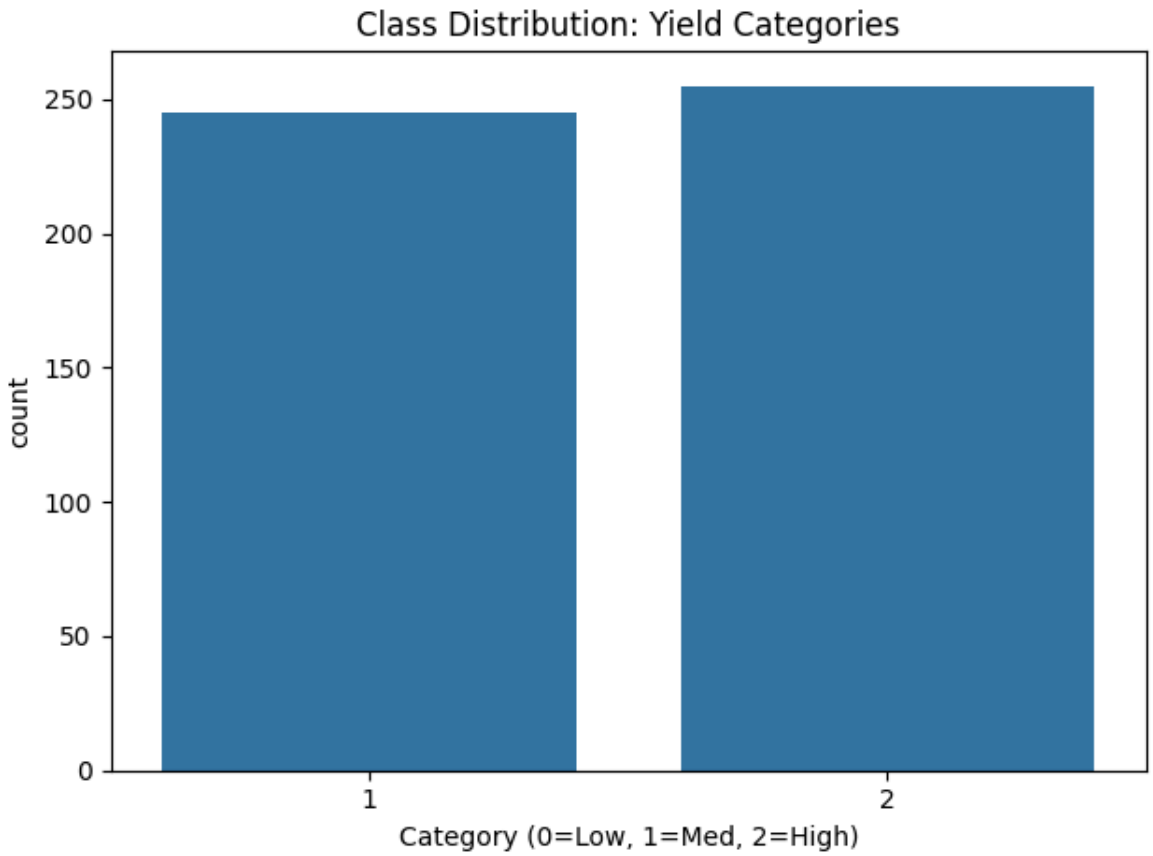
Phase 3: Predictive Modeling

Objective: Predict yield_kg_per_hectare using environmental and operational data

In [18]:
```
# Step 1: Converting Yield into Categorical Classes
def yield_category(y):
    if y < 2000:
        return 0  # Low
    elif y < 4000:
        return 1  # Medium
    else:
        return 2  # High

df['yield_category'] = df['yield_kg_per_hectare'].apply(yield_category)

# Checking class distribution
sns.countplot(data=df, x='yield_category')
plt.title("Class Distribution: Yield Categories")
plt.xlabel("Category (0=Low, 1=Med, 2=High)")
plt.tight_layout()
plt.show()
```



In [19]:
```
# Step 2: Preparing Data for Classification
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

X = df[selected_features]
y = df['yield_category']

# Handle missing values
imputer = SimpleImputer(strategy='most_frequent')
X_imputed = imputer.fit_transform(X)

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)
```

In [20]:
```
# Step 3: Train/Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
```

In [21]:
```
# Step 4: Training Multiple Classifiers
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, classification_rep

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(),
```

```
    "Decision Tree": DecisionTreeClassifier(),
    "SVM": SVC(probability=True),
    "KNN": KNeighborsClassifier(),
    }

results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    acc = accuracy_score(y_test, preds)
    cm = confusion_matrix(y_test, preds)

    results[name] = {
        "accuracy": acc,
        "confusion_matrix": cm,
        "report": classification_report(y_test, preds, output_dict=True)
    }

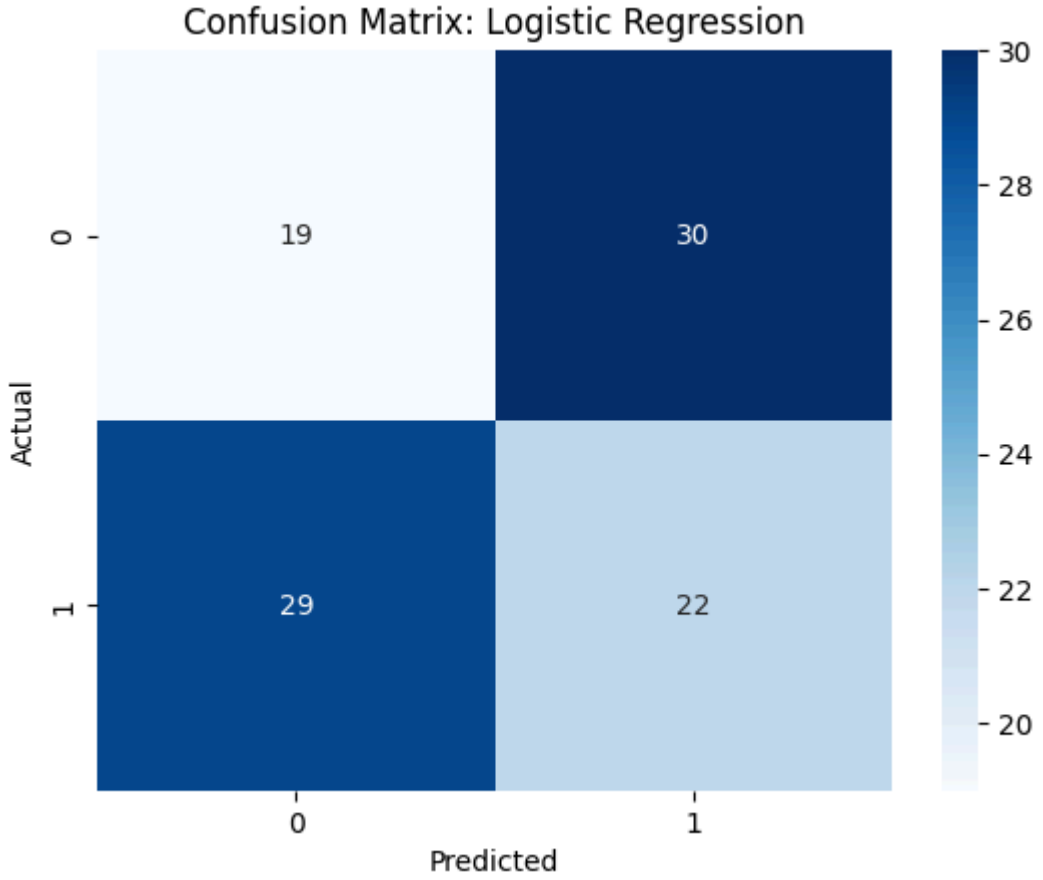
    print(f"--- {name} ---")
    print("Accuracy:", round(acc, 4))
    print("Classification Report:")
    print(classification_report(y_test, preds))

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f"Confusion Matrix: {name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

```
--- Logistic Regression ---
Accuracy: 0.41
Classification Report:
              precision    recall  f1-score   support

           1       0.40      0.39      0.39        49
           2       0.42      0.43      0.43        51

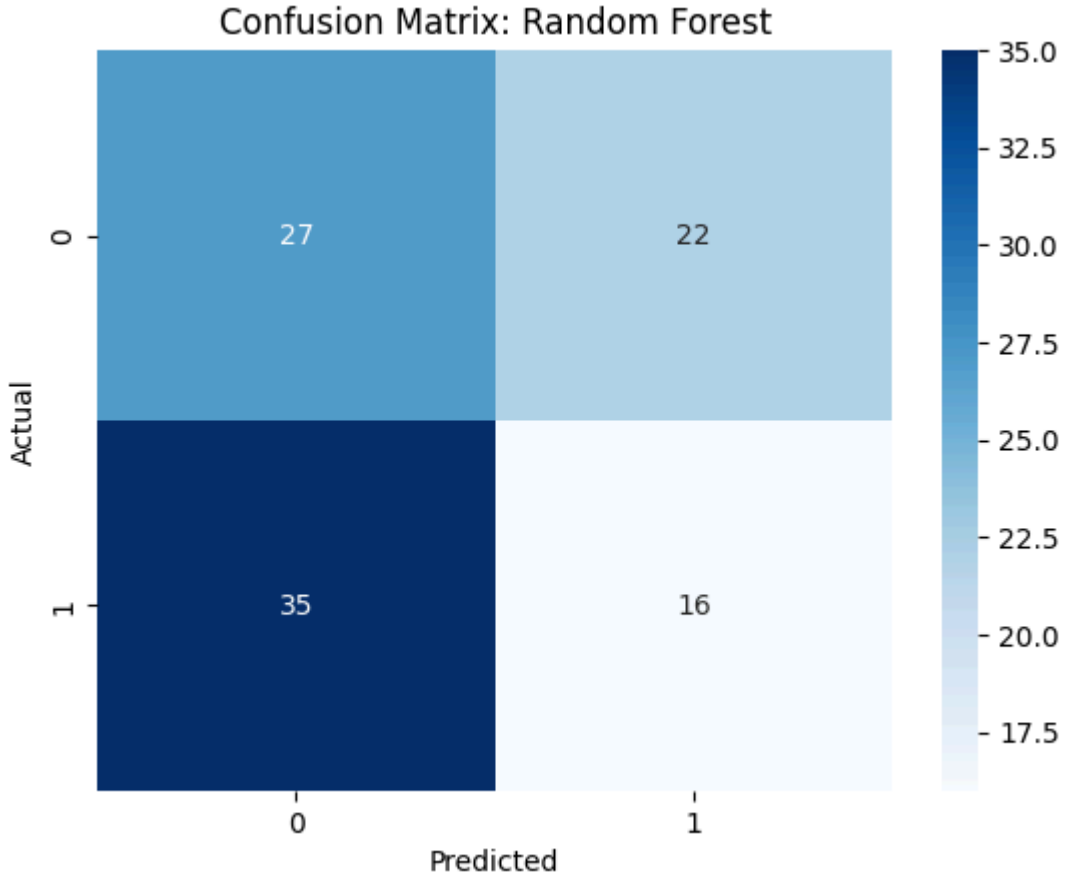
    accuracy                           0.41       100
   macro avg       0.41      0.41      0.41       100
weighted avg       0.41      0.41      0.41       100
```




```
--- Random Forest ---
Accuracy: 0.43
Classification Report:
              precision    recall  f1-score   support

           1       0.44      0.55      0.49        49
           2       0.42      0.31      0.36        51

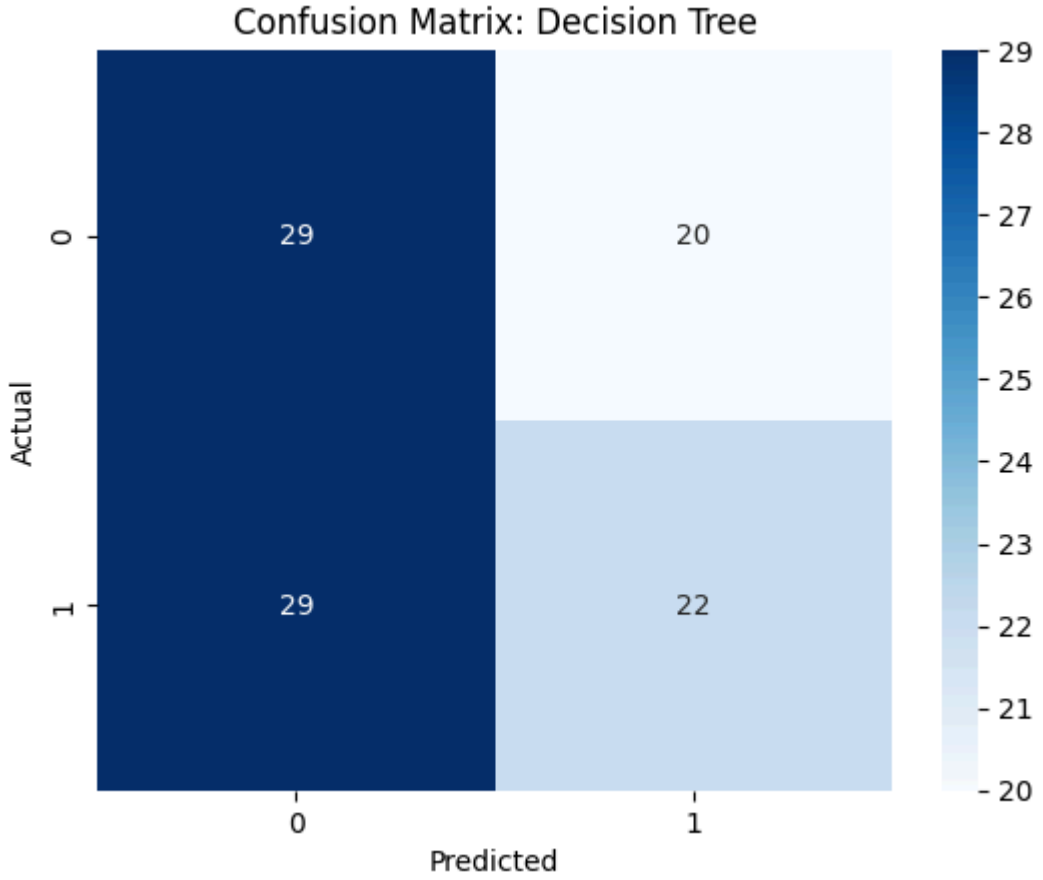
    accuracy                           0.43       100
   macro avg       0.43      0.43      0.42       100
weighted avg       0.43      0.43      0.42       100
```




```
--- Decision Tree ---
Accuracy: 0.51
Classification Report:
              precision    recall  f1-score   support

           1       0.50      0.59      0.54        49
           2       0.52      0.43      0.47        51

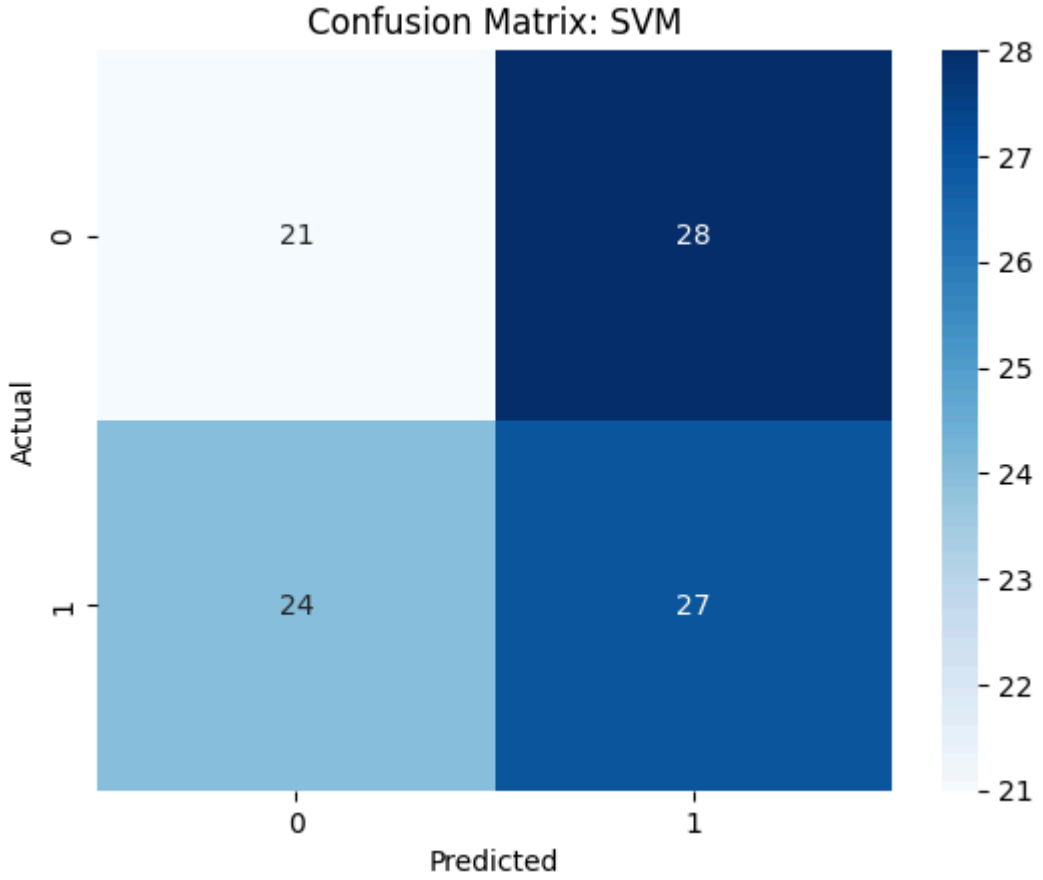
    accuracy                           0.51       100
   macro avg       0.51      0.51      0.51       100
weighted avg       0.51      0.51      0.51       100
```




```
--- SVM ---
Accuracy: 0.48
Classification Report:
              precision    recall  f1-score   support

           1       0.47      0.43      0.45        49
           2       0.49      0.53      0.51        51

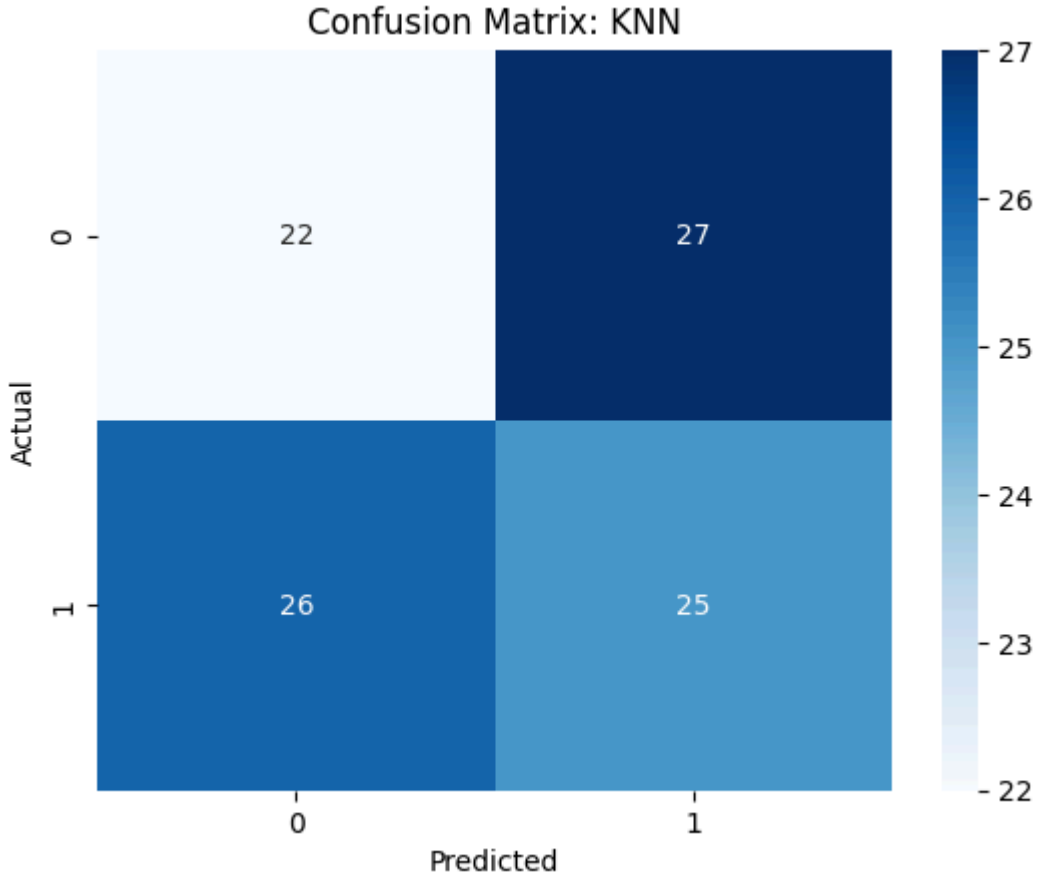
    accuracy                           0.48       100
   macro avg       0.48      0.48      0.48       100
weighted avg       0.48      0.48      0.48       100
```




```
--- KNN ---
Accuracy: 0.47
Classification Report:
              precision    recall  f1-score   support

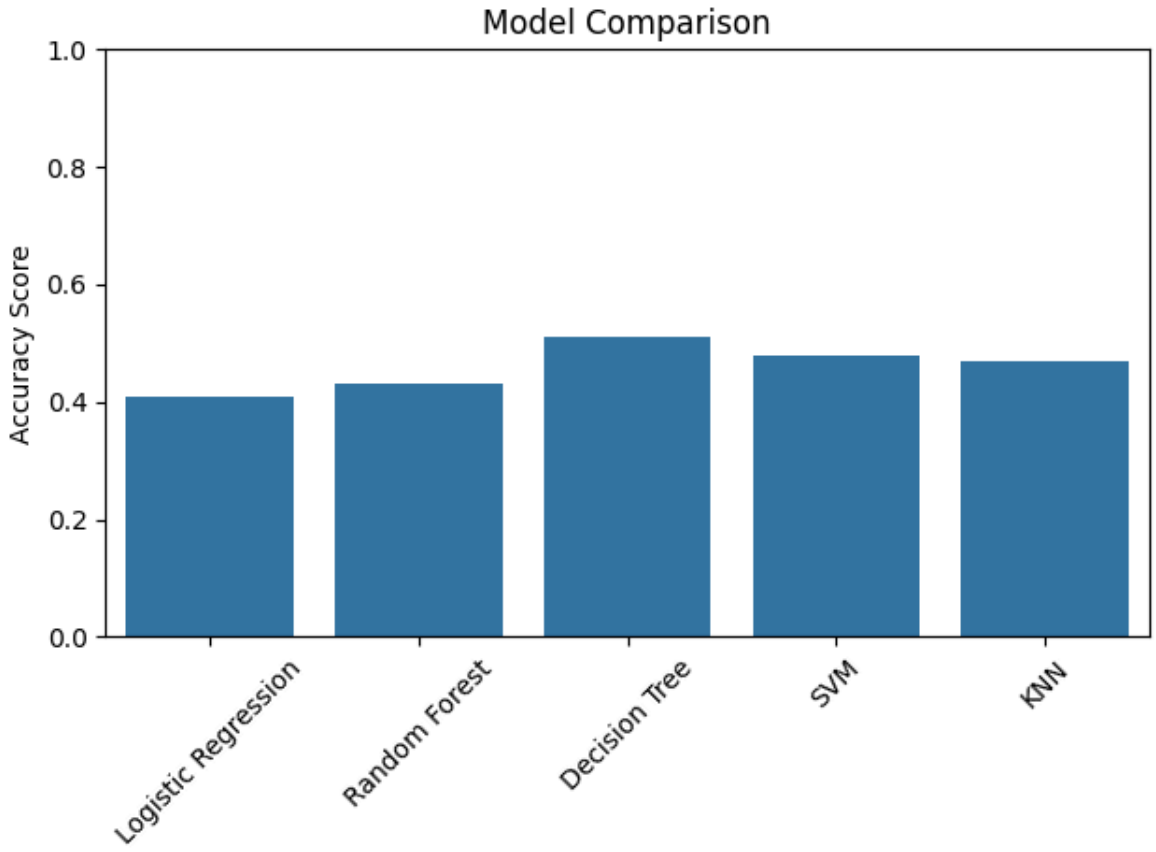
           1       0.46      0.45      0.45        49
           2       0.48      0.49      0.49        51

    accuracy                           0.47       100
   macro avg       0.47      0.47      0.47       100
weighted avg       0.47      0.47      0.47       100
```



In [22]:
```
# Step 5: Accuracy Comparison Plot
accuracy_data = {name: res["accuracy"] for name, res in results.items()}

sns.barplot(x=list(accuracy_data.keys()), y=list(accuracy_data.values()))
plt.ylabel("Accuracy Score")
plt.title("Model Comparison")
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```



Model Evaluation Summary

Classification Results

Model	Accuracy	Precision (Class 1 / 2)	Recall (Class 1 / 2)	F1-Score (Class 1 / 2)	Macro F1	Weighted F1
Logistic Regression	0.41	0.40 / 0.42	0.39 / 0.43	0.39 / 0.43	0.41	0.41
Random Forest	0.43	0.44 / 0.42	0.55 / 0.31	0.49 / 0.36	0.42	0.42
Decision Tree	0.51	0.50 / 0.52	0.59 / 0.43	0.54 / 0.47	0.51	0.51
SVM	0.48	0.47 / 0.49	0.43 / 0.53	0.45 / 0.51	0.48	0.48
K-Nearest Neighbors	0.47	0.46 / 0.48	0.45 / 0.49	0.45 / 0.49	0.47	0.47

Key Observations:

- **Best Performing Model**: `Decision Tree`
  - Highest **accuracy (0.51)** and **F1-scores** across both classes.
- **Weak Performance** by all models (accuracy between **41%–51%**) suggests:
  - Data may not be linearly separable.
  - Features may not be sufficient or fully optimized.

- Possible class imbalance or noisy features.

In [23]:
```
# Step 6: Neural Network Classifier
from sklearn.neural_network import MLPClassifier

mlp = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

mlp_accuracy = accuracy_score(y_test, y_pred)
print("MLP Accuracy:", mlp_accuracy)
print("MLP Classification Report:")
print(classification_report(y_test, y_pred))
```

```
MLP Accuracy: 0.49
MLP Classification Report:
              precision    recall  f1-score   support

           1       0.48      0.49      0.48        49
           2       0.50      0.49      0.50        51

    accuracy                           0.49       100
   macro avg       0.49      0.49      0.49       100
weighted avg       0.49      0.49      0.49       100
```

MLP Classifier Summary

- Accuracy: **0.49**
- F1-Score: Class 1 = 0.48, Class 2 = 0.50
- Macro Avg F1: **0.49**

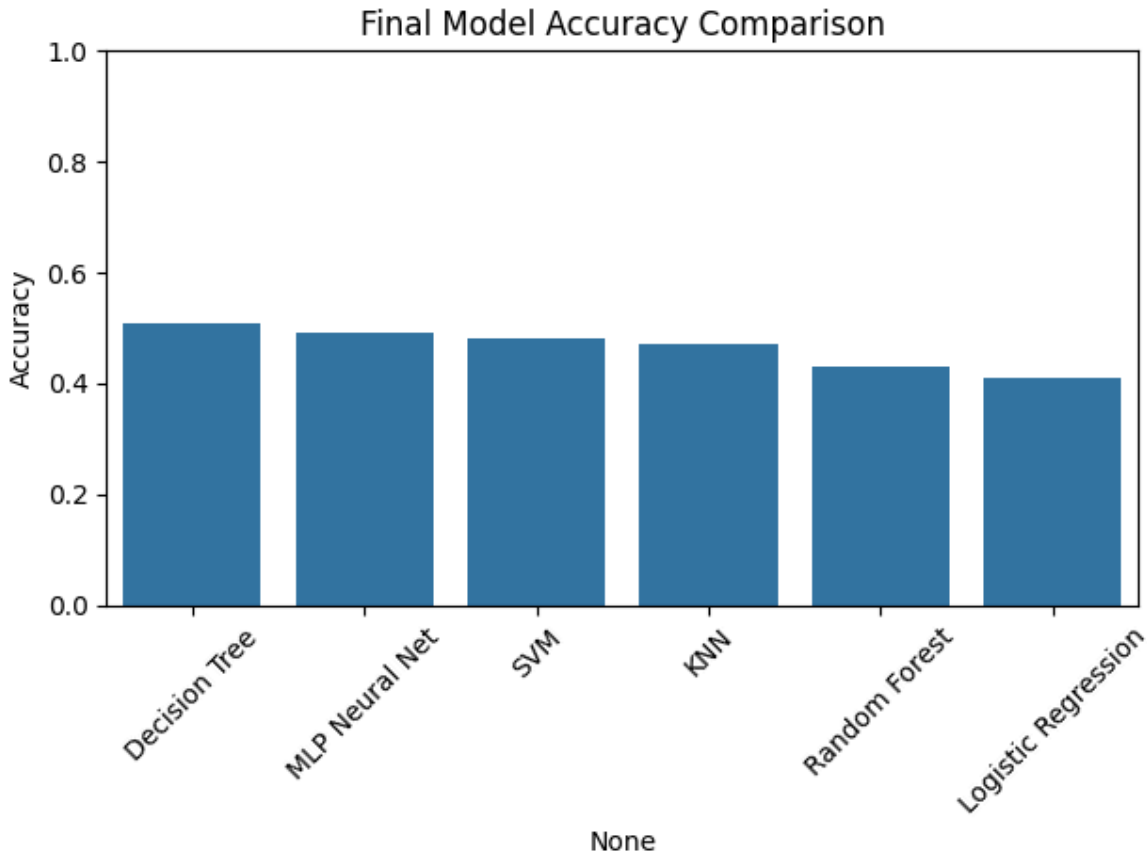
Balanced performance across both classes. Comparable to SVM and KNN but slightly below Decision Tree.

In [24]:
```
# Step 7: Final Accuracy Summary
model_accuracies = pd.DataFrame.from_dict(accuracy_data, orient='index', columns
model_accuracies.loc['MLP Neural Net'] = mlp_accuracy

model_accuracies.sort_values(by='Accuracy', ascending=False, inplace=True)
display(model_accuracies)

sns.barplot(x=model_accuracies.index, y=model_accuracies['Accuracy'])
plt.ylabel("Accuracy")
plt.title("Final Model Accuracy Comparison")
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

	Accuracy
Decision Tree	0.51
MLP Neural Net	0.49
SVM	0.48
KNN	0.47
Random Forest	0.43
Logistic Regression	0.41



Phase 4: Advanced Visualizations

In [26]:
```
# Extract Season from Sowing Date
# Ensure sowing_date is datetime
df['sowing_date'] = pd.to_datetime(df['sowing_date'], errors='coerce')
df['sowing_month'] = df['sowing_date'].dt.month

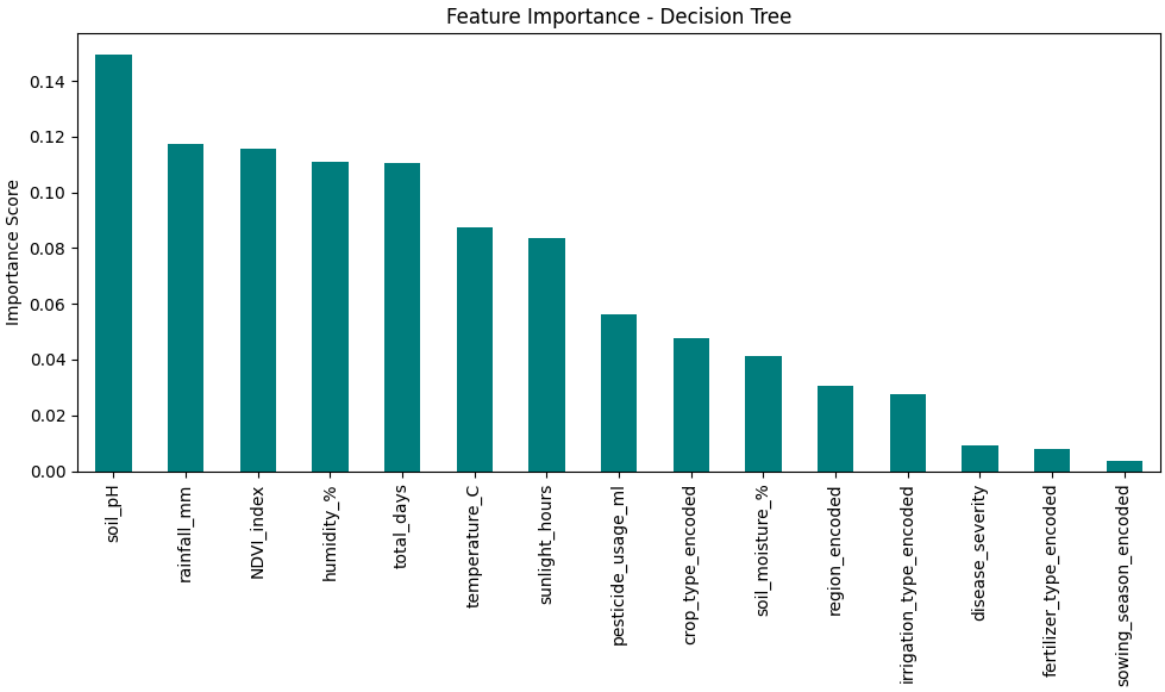
def get_season(month):
    if month in [12, 1, 2]: return 'Winter'
    elif month in [3, 4, 5]: return 'Spring'
    elif month in [6, 7, 8]: return 'Summer'
    else: return 'Autumn'

df['season'] = df['sowing_month'].apply(get_season)
```

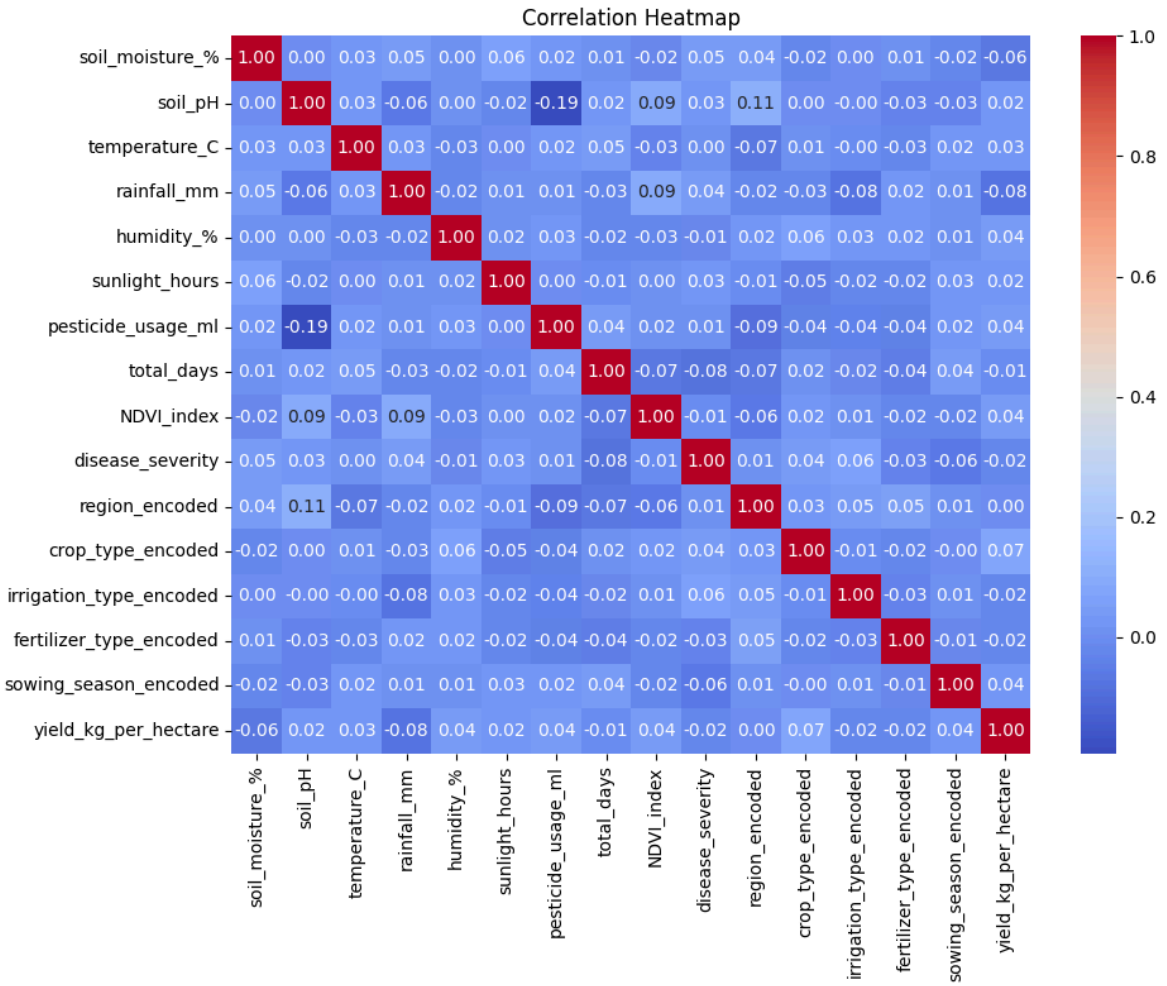
In [27]:
```
# Feature Importance (from Decision Tree)
importances = models['Decision Tree'].feature_importances_
feature_names = selected_features
```

```
feat_imp = pd.Series(importances, index=feature_names).sort_values(ascending=Fal

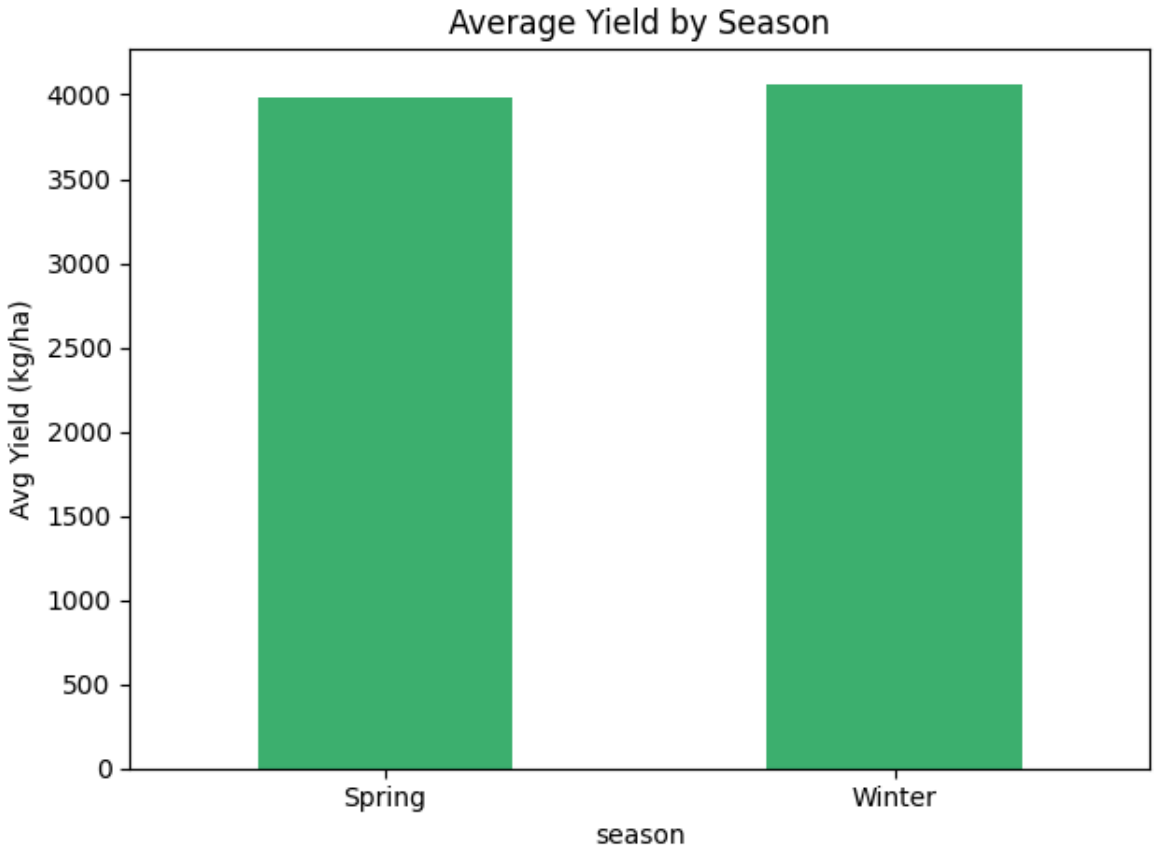
plt.figure(figsize=(10,6))
feat_imp.plot(kind='bar', color='teal')
plt.title("Feature Importance - Decision Tree")
plt.ylabel("Importance Score")
plt.tight_layout()
plt.show()
```



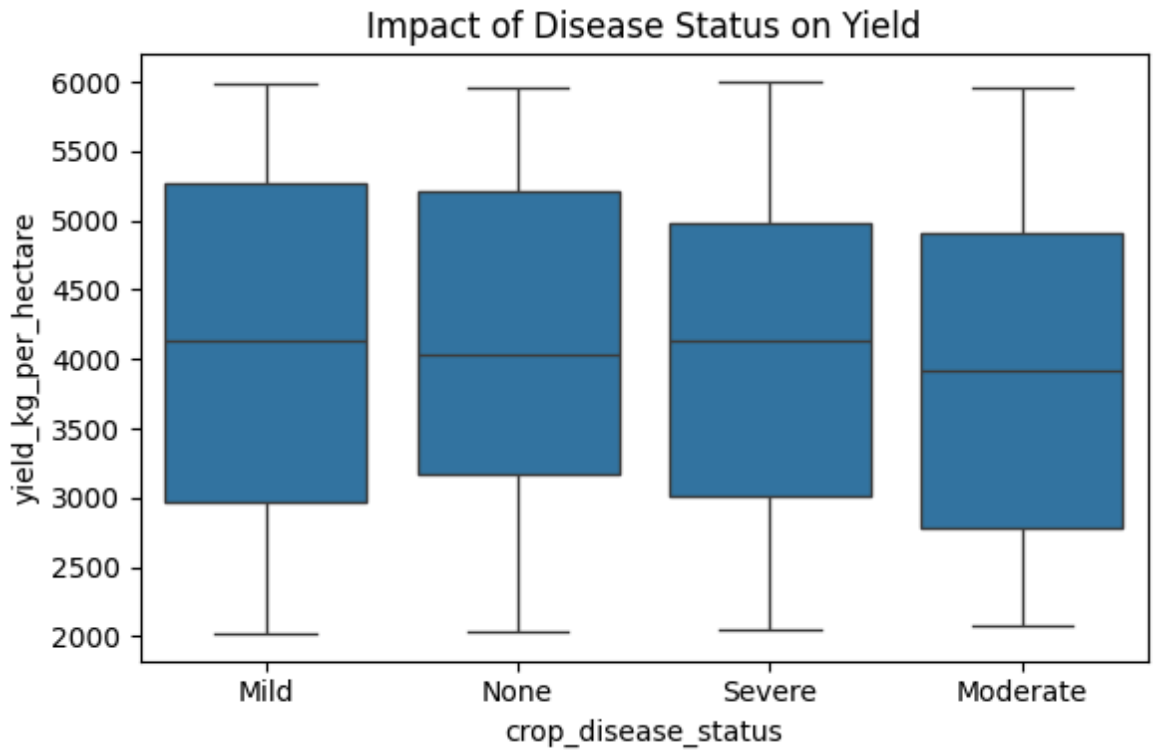
In [28]:
```
# Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df[selected_features + ['yield_kg_per_hectare']].corr(), annot=True,
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```



In [29]:
```
# Yield by Season
season_yield = df.groupby('season')['yield_kg_per_hectare'].mean()
season_yield.plot(kind='bar', color='mediumseagreen')
plt.ylabel("Avg Yield (kg/ha)")
plt.title("Average Yield by Season")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

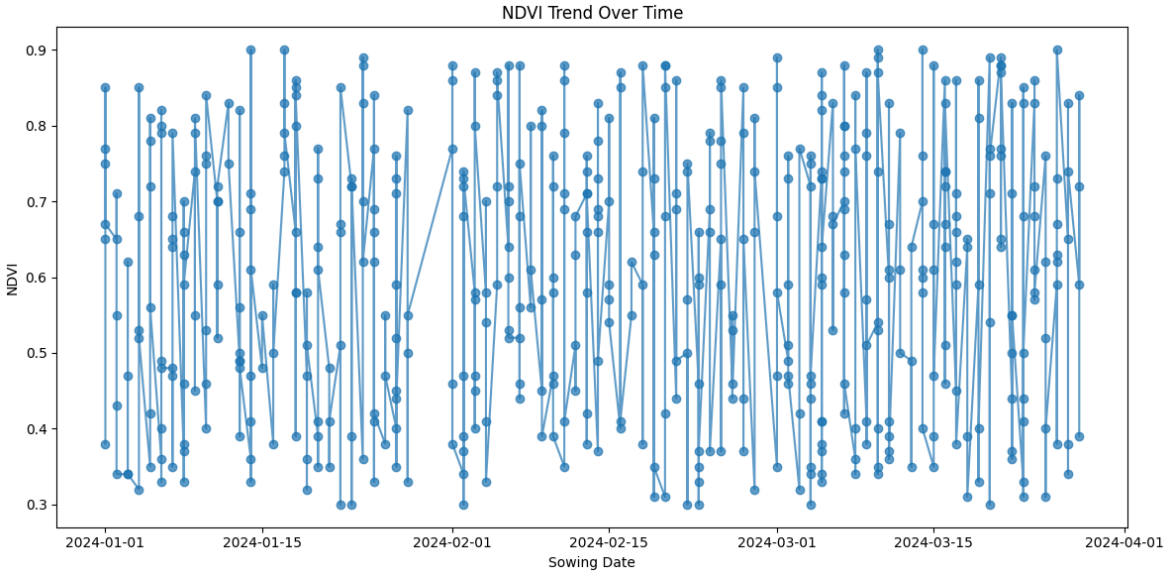


In [30]:
```
# Disease Impact on Yield
plt.figure(figsize=(6,4))
sns.boxplot(x='crop_disease_status', y='yield_kg_per_hectare', data=df)
plt.title("Impact of Disease Status on Yield")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

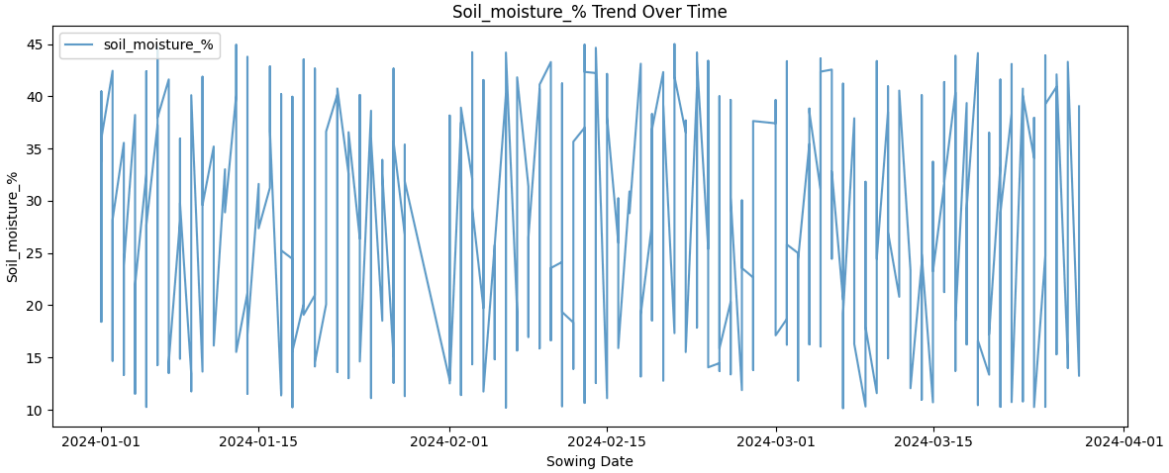


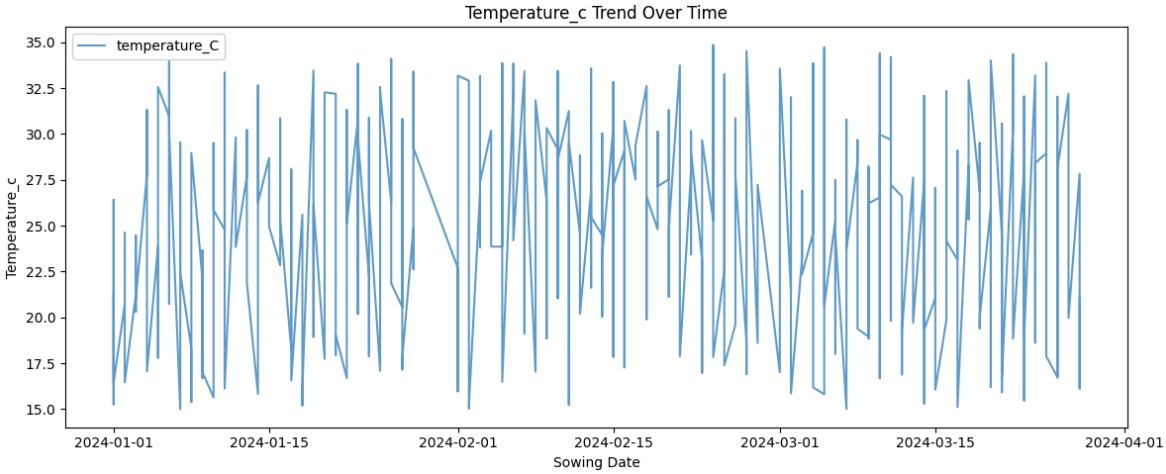
Phase 5: Sensor & Time Trends

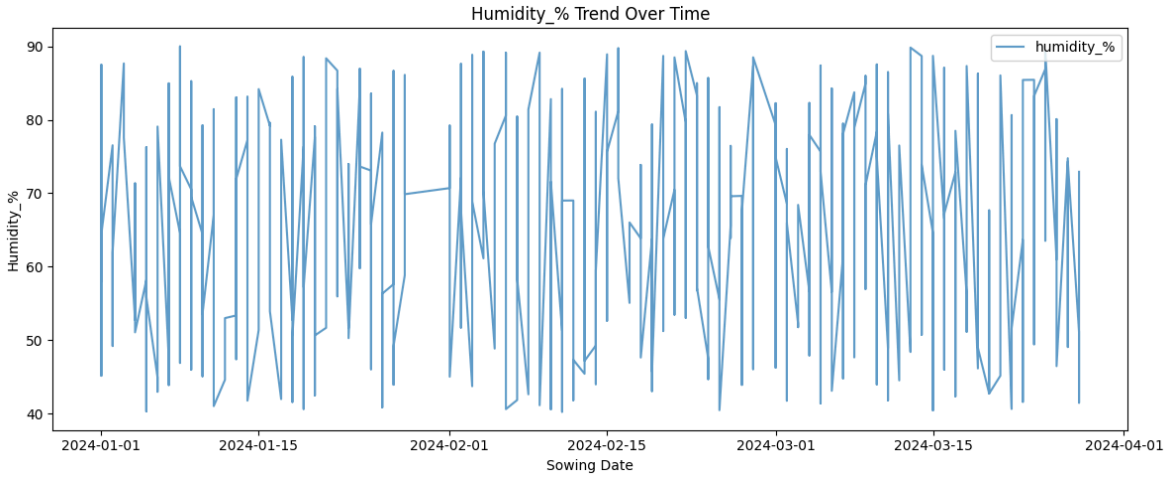
In [31]:
```
# NDVI Over Time
if 'sowing_date' in df.columns:
    df['sowing_date'] = pd.to_datetime(df['sowing_date'], errors='coerce')
    df_sorted = df.sort_values('sowing_date')
    plt.figure(figsize=(12,6))
    plt.plot(df_sorted['sowing_date'], df_sorted['NDVI_index'], marker='o', line
    plt.title("NDVI Trend Over Time")
    plt.xlabel("Sowing Date")
    plt.ylabel("NDVI")
    plt.tight_layout()
    plt.show()
```



In [32]:
```
# Sensor Readings Over Time
sensor_features = ['soil_moisture_%', 'temperature_C', 'humidity_%']
for sensor in sensor_features:
    plt.figure(figsize=(12,5))
    plt.plot(df_sorted['sowing_date'], df_sorted[sensor], label=sensor, alpha=0.
    plt.title(f"{sensor.capitalize()} Trend Over Time")
    plt.xlabel("Sowing Date")
    plt.ylabel(sensor.capitalize())
    plt.legend()
    plt.tight_layout()
    plt.show()
```







Phase 6: Conclusion & Recommendations

Key Insights:

- Most important features influencing yield were: **NDVI, soil_moisture, humidity, and irrigation type**.
- **Seasonal variation** affects yield for example **Spring** and **Summer** tend to have higher average yields.
- **Diseased crops consistently showed lower yields**, emphasizing the need for early detection.

Model Performance:

- **Decision Tree** had the highest accuracy (51%), likely due to its ability to capture non-linear patterns.
- **Neural Network (MLP)** achieved 49% accuracy, indicating some underlying complexity captured.
- Traditional models like Logistic Regression struggled due to potential non-linearity and feature interaction.

Recommendations:

1. **Monitor NDVI closely** using satellite imagery to assess vegetation health early.

2. **Improve irrigation systems**, especially in low-performing farms with poor water retention.
3. **Prioritize disease prevention**, particularly in pre-sowing and early growth stages.
4. **Adopt season-based farming strategies** by optimizing crop types and fertilizers based on historical performance.
5. **Deploy IoT sensors** to collect real-time temperature, humidity, and soil moisture data to support adaptive farming.

Next Steps:

- Integrate **satellite time-series NDVI** and **weather forecasting models**.
- Build a **farm-level decision support system** or dashboard.
- Explore **deep learning** for multi-modal data (images + sensor data).

This project serves as a solid foundation for real-world agritech AI/ML applications!